

Links in the requirements chain

"We just learned that the new union contract is changing how overtime pay and shift bonuses are calculated," Justin reported at the weekly team meeting. "It's also changing how the seniority rules affect priority for vacation scheduling and shift preferences. We have to update the payroll and staff scheduling systems to handle all these changes right away. How long do you think it will take to get this done, Chris?"

"Man, that's going to be a lot of work," said Chris. "The logic for the seniority rules is sprinkled throughout the scheduling system. I can't give you a decent estimate yet. It's going to take hours just to scan through the code and try to find all the places where those rules show up."

Software changes that seem simple often have far-reaching impacts, necessitating modification of many parts of the system. It's hard to find all the system elements that might be affected by an altered requirement. Chapter 28, "Change happens," discussed the importance of performing an impact analysis to make sure the team knows what it's getting into before it commits to implementing a proposed change. Change impact analysis is easier if you have a road map that shows where each requirement or business rule was implemented in the software.

This chapter addresses the subject of requirements tracing (or traceability). Requirements trace information documents the dependencies and logical links between individual requirements and other system elements. These elements include other requirements of various types, business rules, architecture and other design components, source code modules, tests, and help files. Trace information facilitates impact analysis by helping you identify all the work products you might have to modify to implement a proposed requirement change.

Tracing requirements

Trace links allow you to follow the life of a requirement both forward and backward, from origin through implementation. Chapter 11, "Writing excellent requirements," identified traceability as one of the characteristics of excellent requirements. (Note that being *traceable*—having the properties to facilitate tracing—is not the same as being *traced*—actually having logical links between requirements and other elements recorded.) For requirements to be traceable, each one must be uniquely and persistently labeled so that you can refer to it unambiguously throughout the project. Write the requirements in a fine-grained fashion, rather than creating large paragraphs containing many individual functional requirements that readers have to parse out.

Figure 29-1 illustrates four types of requirements trace links (Jarke 1998). Customer needs are traced *forward to requirements*, so you can tell which requirements will be affected if those needs change during or after development. Customer needs could be articulated in the form of business objectives, market demands, and/or user requirements. A complete set of forward traces also gives you confidence that the requirements set has addressed all stated customer needs. Conversely, you can trace *backward from requirements* to customer needs to identify the origin of each software requirement. If you choose to represent customer needs in the form of use cases, the top half of Figure 29-1 illustrates tracing between use cases and functional requirements.

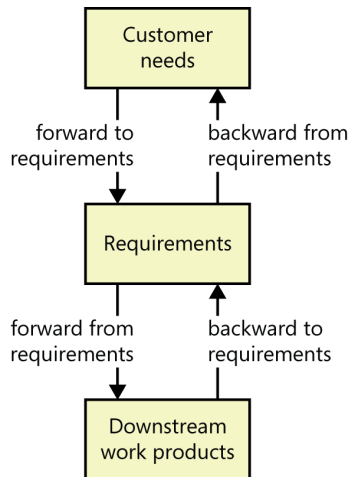


FIGURE 29-1 Four types of requirements tracing.

The bottom half of Figure 29-1 indicates that, as requirements flow into downstream deliverables during development, you can trace *forward from requirements* by defining links between individual functional and nonfunctional requirements and specific system elements. This type of link allows you to determine that you’ve satisfied every requirement because you know which design components and code elements address each one. The fourth type of link traces specific product elements *backward to requirements* so that you know why each element was created. Most applications include some scaffolding or enabling code, such as for testing, that doesn’t relate directly to user-specified requirements, but you should know why each line of code was written.

Suppose a tester encounters unexpected functionality with no corresponding written requirement. This code could indicate that a developer implemented a legitimate implied or verbally communicated requirement that the business analyst can now add to the requirements set. Alternatively, it might be “orphan code,” an instance of gold-plating that doesn’t belong in the product. Trace links can help you sort out these kinds of situations and build a more complete picture of how the pieces of your system fit together. Conversely, tests that are derived from—and traced back to—individual requirements provide a mechanism for detecting unimplemented requirements, because the expected functionality will be missing from the system being tested. Trace links also help you keep track of parentage, interconnections, and dependencies among individual requirements.

This information reveals the propagation of change that can result when a particular requirement is deleted or modified.

Figure 29-2 illustrates many kinds of traceability relationships that can be defined on a project. Of course, you don't need to define and manage all these trace link types. On many projects, you can gain most of the traceability benefits you want for just a fraction of the potential effort. Maybe you only need to trace system tests back to functional requirements or user requirements. Perform a cost-benefit analysis to decide which links will contribute to the success of your project, both in terms of development and long-term maintenance effort. Don't ask team members to spend time recording information unless you know how they can use it.

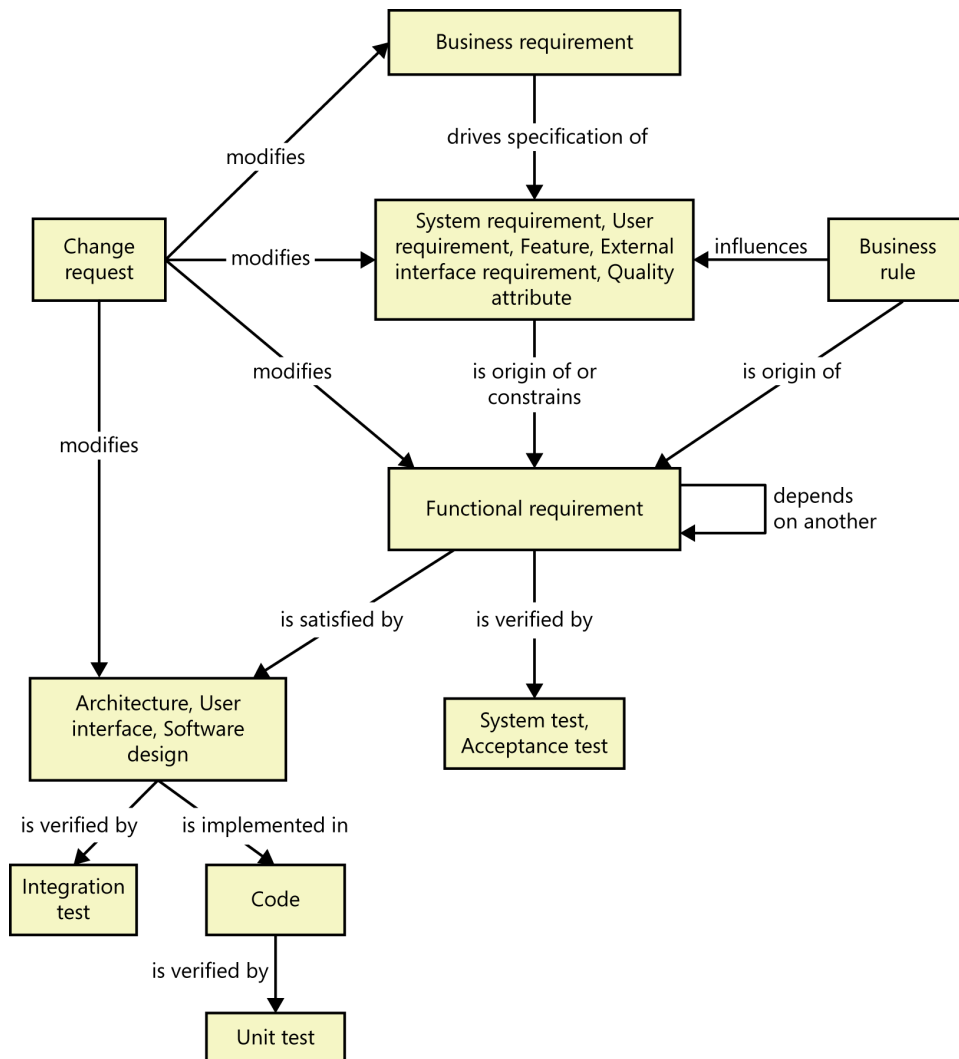


FIGURE 29-2 Some possible requirements trace links.

Motivations for tracing requirements



I've had the embarrassing experience of writing a program and then realizing that I had inadvertently overlooked a requirement. It was in the SRS—I simply missed it. I had to go back and write additional code after I thought I was done programming. Overlooking a requirement is more than an embarrassment if it means a customer isn't satisfied or a product is missing a critical function. Requirements tracing provides a way to demonstrate compliance with a specification, contract, or regulation. At an organization level, implementing requirements tracing can improve the quality of your products, reduce maintenance costs, and facilitate reuse.

Keeping the link information current as the system undergoes development and maintenance takes discipline and time. If the trace information becomes obsolete, you'll probably never reconstruct it. Obsolete or inaccurate trace data wastes time by sending developers and maintainers down the wrong path, destroying any trust the developers might have had in the information. Because of these realities, you should adopt requirements tracing for the right reasons (Ramesh et al. 1995). Following are some potential benefits of implementing requirements tracing:

- **Finding missing requirements** Look for business requirements that don't trace to any user requirements, and user requirements that don't trace to any functional requirements.
- **Finding unnecessary requirements** Look for any functional requirements that don't trace back to user or business requirements and therefore might not be needed.
- **Certification and compliance** You can use trace information when certifying a safety-critical product, to demonstrate that all requirements were implemented—although that doesn't confirm that they were implemented correctly! Trace information demonstrates that requirements demanded for regulatory compliance have been included and addressed, as is often needed for applications for health care and financial services companies.
- **Change impact analysis** Without trace information, there's a good chance that you'll overlook a system element that would be affected if you add, delete, or modify a particular requirement.
- **Maintenance** Reliable trace information facilitates your ability to make changes correctly and completely during maintenance. When corporate policies or government regulations change, software systems often must be updated. A table that shows where each applicable business rule was addressed in the functional requirements, designs, and code makes it easier to make the necessary changes properly.
- **Project tracking** If you record the trace data during development, you'll have an accurate record of the implementation status of planned functionality. Absent links indicate work products that have not yet been created.
- **Reengineering** You can list the functions in an existing system you're replacing and trace them to where they are addressed in the new system's requirements and software components.

- **Reuse** Trace information facilitates the reuse of product components by identifying packages of related requirements, designs, code, and tests.
- **Testing** When a test fails, the links between tests, requirements, and code point developers toward likely areas to examine for the defect.

Many of these are long-term benefits, reducing overall product life-cycle costs but increasing the development cost by the effort expended to accumulate and manage the trace information. View requirements tracing as an investment that increases your chances of delivering a maintainable product that satisfies all the stated customer requirements. This investment will pay dividends anytime you have to modify, extend, or replace the product. Establishing traces is not much work if you collect the information as development proceeds, but it's tedious and expensive to do on a completed system.

The requirements traceability matrix



The most common way to represent the links between requirements and other system elements is in a *requirements traceability matrix*, also called a *requirements trace matrix* or a *traceability table*. Joy Beatty and Anthony Chen (2012) describe a similar tool called a *requirements mapping matrix* that shows the relationships between multiple types of objects. Table 29-1 illustrates a portion of a requirements traceability matrix, drawn from the Chemical Tracking System. When I've set up such matrices in the past, I started with a copy of the baselined SRS and deleted everything except the labels for the functional requirements. Then I set up a table laid out like Table 29-1 with only the "Functional requirement" column populated. As fellow team members and I worked on the project, we gradually filled in the blank cells in the matrix.

TABLE 29-1 One kind of requirements traceability matrix

User requirement	Functional requirement	Design element	Code element	Test
UC-28	catalog.query.sort	Class catalog	CatalogSort()	search.7 search.8
UC-29	catalog.query.import	Class catalog	CatalogImport() CatalogValidate()	search.12 search.13 search.14

Table 29-1 shows how each functional requirement is linked backward to a specific use case and forward to one or more design, code, and test elements. A design element can be something like an architectural component, a table in a relational data model, or an object class. Code references can be class methods, stored procedures, source code file names, or modules within a source file. Including more trace detail takes more work, but it gives you the precise locations of the related software elements.

Fill in the information as the work gets done, not as it gets planned. That is, enter **CatalogSort()** in the "Code element" column of the first row in Table 29-1 only when the code in that function has been written. That way a reader knows that populated cells in the requirements traceability matrix indicate work that's been completed.



Important Listing the test cases for each requirement does *not* indicate that the software has passed those tests. It simply indicates that certain tests have been written to verify the requirement at the appropriate time. Tracking testing status is a separate matter.

Another way to represent trace information is through a set of matrices that define links between pairs of system elements, such as these:

- One type of requirement to other requirements of that same type
- One type of requirement to requirements of another type
- One type of requirement to tests

You can use these matrices to define various relationships that are possible between pairs of requirements, such as “specifies/is specified by,” “is dependent on,” “is parent of,” and “constrains/is constrained by” (Sommerville and Sawyer 1997).

Table 29-2 illustrates a two-way traceability matrix. Most cells in the matrix are empty. Each cell at the intersection of two linked components contains a symbol to indicate the connection. Table 29-2 uses an arrow to indicate that a certain functional requirement is traced from a particular use case. For instance, FR-2 is traced from UC-1, and FR-5 is traced from both UC-2 and UC-4. This indicates that the functional requirement FR-5 is reused across two use cases, UC-2 and UC-4.

TABLE 29-2 Requirements traceability matrix showing links between use cases and functional requirements

Functional requirement	Use case			
	UC-1	UC-2	UC-3	UC-4
FR-1	↙			
FR-2	↙			
FR-3			↙	
FR-4			↙	
FR-5		↙		↙
FR-6			↙	

Trace links can define one-to-one, one-to-many, or many-to-many relationships between system elements. The format in Table 29-1 accommodates these cardinalities by letting you enter several items in each table cell. Here are some examples of the possible link cardinalities:

- **One-to-one** One design element is implemented in one code module.
- **One-to-many** One functional requirement is verified by multiple tests.

- **Many-to-many** Each use case leads to multiple functional requirements, and certain functional requirements are common to several use cases. Similarly, a shared or repeated design element might satisfy several functional requirements. Ideally, you'll capture all these interconnections, but in practice, many-to-many trace relationships become complex and difficult to manage.

Nonfunctional requirements such as quality attributes often do not trace directly into code. A response-time requirement might dictate the use of certain hardware, algorithms, database structures, and architectural approaches. A portability requirement could restrict the language features that the programmer uses but might not result in specific code segments that enable portability. Other quality attributes are indeed implemented in code. Security requirements for user authentication lead to derived functional requirements that might be implemented through passwords or biometrics functionality. In those cases, you can trace the corresponding functional requirements backward to their parent nonfunctional requirement and forward into downstream deliverables as usual. Figure 29-3 illustrates a possible traceability chain involving nonfunctional requirements.

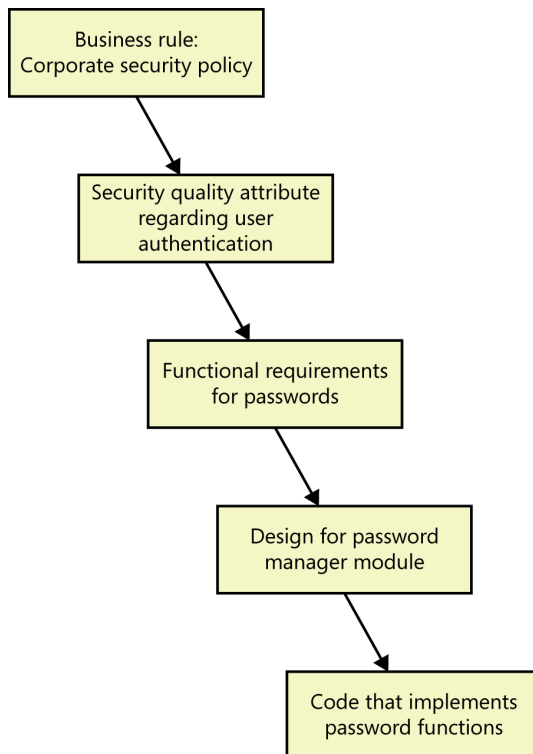


FIGURE 29-3 Sample traceability chain for requirements dealing with application security.

Trace links should be defined by whomever has the appropriate information available. Table 29-3 identifies some typical sources of knowledge about links between various types of source and target objects. Determine the roles and individuals who should supply each type of trace information for your project. Expect some pushback from busy people whom the analyst or project manager asks

to provide this data. Those practitioners are entitled to an explanation of requirements tracing, why it provides value, and why they're being asked to contribute to the process. Point out that the incremental cost of capturing trace information at the time the work is done is small; it's primarily a matter of habit, discipline, and having the storage mechanism established.

Trap Gathering and managing requirements trace data must be made the explicit responsibility of certain individuals or it won't happen. Typically, a business analyst or a quality assurance engineer collects, stores, and reports on the trace information.

TABLE 29-3 Likely sources of trace link information

Link source object type	Link target object type	Information source
System requirement	Functional requirement	System engineer
User requirement	Functional requirement	Business analyst
Business requirement	User requirement	Business analyst
Functional requirement	Functional requirement	Business analyst
Functional requirement	Test	Tester
Functional requirement	Architecture element	Architect or developer
Functional requirement	Other design elements	Designer or developer
Design element	Code	Developer
Business rule	Functional requirement	Business analyst

Tools for requirements tracing

As Chapter 30, “Tools for requirements engineering,” describes, commercial requirements management tools often have powerful requirements-tracing capabilities. You can store requirements and other information in a tool’s database and define links between the various types of stored objects, including peer links between two requirements of the same kind. Some tools let you differentiate traced-to and traced-from relationships, automatically defining the complementary links. That is, if you indicate that requirement R is *traced to* test T, the tool will also show the symmetrical relationship in which T is *traced from* R.

Some tools automatically flag a trace link as being *suspect* whenever the object on either end of the link is modified. A suspect link displays a visual indicator (such as a red question mark or a diagonal red line) in the corresponding cell in the requirements traceability matrix. For example, if you changed Use Case 3, the requirements traceability matrix in Table 29-2 might look like Table 29-4 the next time you see it. The suspect link indicators (in this case, question marks) tell you to check whether functional requirements 3, 4, and 6 need to be changed to remain consistent with the modified UC-3. After making any necessary changes, you clear the suspect link indicators manually. This process helps ensure that you’ve accounted for the known ripple effects of a change.

TABLE 29-4 Suspect links in a requirements traceability matrix

Functional requirement	Use case			
	UC-1	UC-2	UC-3	UC-4
FR-1	↙			
FR-2	↙			
FR-3			↙?	
FR-4			↙?	
FR-5		↙		↙
FR-6			↙?	



Requirements management tools also let you define cross-project or cross-subsystem links. I know of one large software product that had 20 major subsystems, with certain high-level system requirements apportioned among multiple subsystems. In some cases, a requirement that was allocated to one subsystem was actually implemented through a service that another subsystem provided. This project used a requirements management tool to successfully track these complex trace relationships.

It's impossible to perform requirements tracing manually for any but very small applications. You can use a spreadsheet to maintain trace data for up to a couple hundred requirements, but larger systems demand a more robust solution. Requirements tracing can't be fully automated because the knowledge of the links originates in the development team members' minds. However, after you've identified the links, tools can help you manage the vast quantity of trace information.

A requirements tracing procedure

Consider following this sequence of steps when you begin to implement requirements tracing on a specific project:

1. Educate the team and your management about the concepts and importance of requirements tracing, your objectives for this activity, where the trace data is stored, and the techniques for defining the links. Ask all participants to commit to their responsibilities.
2. Select the link relationships you want to define from the possibilities shown in Figure 29-2. Don't try to do all of these at once! You'll be overwhelmed.
3. Choose the type of traceability matrix you want to use: the single-matrix style shown in Table 29-1 or several matrices like the one illustrated in Table 29-2. Select a mechanism for

storing the data: a table in a text document, a spreadsheet, or (much better) a requirements management tool.

4. Identify the parts of the product for which you want to maintain traceability information. Start with the critical core functions, the high-risk portions, or the portions that you expect will undergo the most maintenance and evolution over the product's life.
5. Identify the individuals who will supply each type of link information and the person (most likely a BA) who will coordinate the tracing activities and manage the data.
6. Modify your development procedures to remind developers to update the links after implementing a requirement or an approved change. The trace data should be updated soon after someone completes a task that creates or changes a link in the requirements chain.
7. Define the labeling conventions you will use to give each system element a unique identifier so that the elements can be linked together. Chapter 10, "Documenting the requirements," described several ways to label requirements.
8. As development proceeds, have each participant provide the requested trace information as they complete small bodies of work. Stress the advantage of ongoing accumulation of the trace data over assembling it at a major milestone or at the end of the project.
9. Audit the trace information periodically to make sure it's being kept current. If a requirement is reported as implemented and verified, yet its trace data is incomplete or inaccurate, your requirements tracing process isn't working as intended.

I've described this procedure as though you were starting to collect trace information at the outset of a new project. If you're maintaining an existing system, you probably don't have trace data available. There's no time like the present to begin accumulating this information. The next time you add an enhancement or make a modification, write down what you discover about connections between code, tests, designs, and requirements. You'll never reconstruct a complete requirements traceability matrix, but this small amount of effort might make it easier the next time someone needs to work on that same part of the system.



Best approached with caffeine and music

My friend Sonoko, a highly experienced software developer who works on credit-card transaction processing systems, recently sent me an email message. "I thought you'd be amused to know that I've spent the afternoon creating a requirements traceability matrix for one of my projects, and I'm about to die of tedium," Sonoko said. "The requirements spec was 30 pages long, my technical design is 100 pages long, and the matrix is therefore hefty. I know that we have to do them, but I fell asleep two hours ago."

I asked Sonoko some follow-up questions to better understand what she was doing. "Since I make my technical designs available to the business analyst, affected business areas, and project manager, the traceability matrix proves to them that I addressed every requirement they gave me," she replied. "In my design review, I present the design by walking through the

traceability matrix, which is logically sequenced by requirement.” I asked Sonoko why she was taking the time to create this traceability matrix. She said, “I create it because it ensures that I cover everything, and it provides a quick way for me to see all of the system elements that a given requirement affects.”

After working for decades in the software industry, Sonoko clearly understands the value that linking requirements to affected design elements can provide. But, as she points out, it’s not a fun chore to wade through such a large volume of information and link the bits together. If the way she approaches technical design permits, it would save time to begin aggregating the trace information as her design begins to stabilize, instead of at the end.

Is requirements tracing feasible? Is it necessary?

You might conclude that accumulating requirements trace information is more expensive than it’s worth or that it’s not feasible for your project. That’s entirely possible. Acquiring a tool with the necessary capabilities, setting it up, entering the data, and keeping it current is expensive and time consuming. You might not need to construct a group memory like this if members of your team possess the necessary knowledge and share it with others when it’s needed. Only your team can decide whether requirements tracing—be it just requirements-to-tests or something more elaborate—adds value to your project above its cost.



Consider the following example, though. A conference attendee who worked at an aircraft manufacturer told me that the SRS for his team’s part of the company’s latest jetliner was a stack of paper six feet thick. They had a complete requirements traceability matrix. I’ve flown on that very model of airplane, and I was happy to hear that the developers had managed their software requirements so carefully. Managing traces on a huge product with many interrelated subsystems is a lot of work. This aircraft manufacturer knows it is essential. The U.S. Federal Aviation Administration agrees: traceability from requirements to designs is required for certification of aviation software. Similarly, the U.S. Food and Drug Administration advocates that medical device manufacturers demonstrate traceability of a product’s requirements into downstream deliverables as part of the validation process for the device.



Even if your products won’t cause loss of life or limb if they fail, you should take requirements tracing seriously. At a minimum, consider tracing between business requirements and user requirements to look for alignment, omissions, and unnecessary requirements. The CEO of a major corporation who was present when I described requirements tracing at a seminar asked, “Why *wouldn’t* you do this for your strategic business systems?” That’s an excellent question. You should decide to use any improved requirements engineering practice based on both the costs of applying the technique and the risks of *not* using it. As with all software processes, make an economic decision to invest your valuable time where you expect the greatest payback.



Next steps

- Set up a trace matrix for 15 or 20 requirements from an important portion of the system you're currently developing. Try the approaches shown in both Tables 29-1 and 29-2. Populate the matrix as the project progresses for a few weeks. Evaluate which method seems most effective and what procedures for collecting and storing traceability information will work for your team.
- The next time you perform maintenance on a poorly documented system, record what you learn from reverse engineering the part of the product you're modifying. Build a fragment of a requirements traceability matrix for the piece of the puzzle you're manipulating so that the next time someone has to work on it they have a head start. Grow the matrix as your team continues to maintain the product.
- Trace your functional requirements back to user requirements, and trace your user requirements to business requirements. Count the requirements that you could cut because they don't link back to a business requirement. Count the requirements that were missing until the trace matrix revealed their absence. Estimate the costs had you not discovered these requirements errors until much later in the project. This analysis will help you judge whether requirements tracing will pay off in your environment.